

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

**JOÃO ANTONIO CARBONI GOMES
DIOGO CALEGARI DOS REIS**

**ATIVIDADE PRÁTICA: SISTEMA DE CADASTRO DE ALUNOS
UTILIZANDO ESTRUTURAS DE ÁRVORES**

**CORNÉLIO PROCÓPIO
2026**

ATIVIDADE PRÁTICA: SISTEMA DE CADASTRO DE ALUNOS UTILIZANDO ESTRUTURAS DE ÁRVORES

Matéria: Estrutura de Dados 2026-01

Professor: Clayton Kossoski

1. Estrutura de Árvore Escolhida

Para a implementação do sistema de cadastro de alunos, optamos pela Árvore Binária de Busca (ABB). Essa escolha se justifica pela adequação direta ao problema proposto: a ABB permite inserção, busca e listagem ordenada de forma eficiente, com implementação relativamente simples e código de fácil compreensão e manutenção.

A chave de ordenação utilizada é o nome completo do aluno (string). Assim, para cada nó da árvore, todos os nós à esquerda possuem nomes lexicograficamente menores, e todos à direita possuem nomes lexicograficamente maiores. Essa propriedade garante que um percurso in-order (esquerda - raiz - direita) produza automaticamente a listagem em ordem alfabética.

Cada nó da árvore representa um aluno e armazena os seguintes campos:

- matricula (int): identificador único do aluno
- nome (string): nome completo, utilizado como chave de ordenação
- curso (string): curso em que o aluno está matriculado
- esquerda (Aluno*): ponteiro para o filho com nome menor
- direita (Aluno*): ponteiro para o filho com nome maior

2. Operações Implementadas

2.1 Inserção

Função: **inserir(raiz, matricula, nome, curso)**

A inserção é realizada de forma recursiva. O algoritmo compara o nome do novo aluno com o nome do nó atual: se menor, desce para a subárvore esquerda; se maior, desce para a subárvore direita. Quando encontra uma posição nula, cria um novo nó. Caso o nome já exista na árvore, o sistema exibe uma mensagem informando que o aluno já está cadastrado, evitando duplicatas.

2.2 Busca

Função: **buscar(raiz, nome)**

A busca também é recursiva. Em cada nó, o nome buscado é comparado com o nome armazenado: se igual, o nó é retornado; se menor, a busca prossegue à esquerda; se maior, à direita. Caso se chegue a um nó nulo, retorna nullptr, indicando que o aluno não foi encontrado.

2.3 Listagem em Ordem Alfabética

Função: **listarEmOrdem(raiz)**

Implementada por meio de um percurso in-order: visita primeiro a subárvore esquerda, depois exibe os dados do nó atual, e por fim visita a subárvore direita. Por definição da ABB, esse percurso garante que os alunos sejam listados em ordem crescente de nome, sem necessidade de qualquer algoritmo de ordenação externo.

2.4 Informações da Árvore

Funções: **contarAlunos(raiz)** e **altura(raiz)**

A contagem de alunos é feita recursivamente: retorna 0 para árvore vazia ou 1 + (contagem da subárvore esquerda) + (contagem da subárvore direita). O cálculo da altura também é recursivo: compara as alturas das duas subárvores e retorna a maior delas acrescida de 1.

3. Exemplo de Execução

Inserindo os alunos do exemplo do enunciado na seguinte ordem: John Smith (101), Alice Johnson (102), Michael Brown (103) e Emily Davis (104), a árvore formada tem a seguinte estrutura:

101	João Carboni	Análise e Desenvolvimento de Sistemas
102	Diogo Calegari	Análise e Desenvolvimento de Sistemas
103	Gabriel Pirolo	Engenharia de Software
104	Cláudio Marengoni	Análise e Desenvolvimento de Sistemas

A listagem in-order produz a saída esperada em ordem alfabética:

- Cláudio Marengoni - Matrícula: 104 - Curso: Análise e Desenvolvimento de Sistemas
- Diogo Calegari - Matrícula: 102 - Curso: Análise e Desenvolvimento de Sistemas
- Gabriel Pirolo - Matrícula: 103 - Curso: Engenharia de Software
- João Carboni - Matrícula: 103 - Curso: Análise e Desenvolvimento de Sistemas